ApexQuant v10.0 – Master Build Prompt

Football & Tennis Quantitative Analysis Suite – organized, professional version

Is document ka istemal: Neeche wala poora prompt (Section 1 se Section 12 tak) copy karke kisi bhi coding AI (Claude, etc.) ko do. Prompt English mein hai taake AI exact samjhe; har section ke upar Roman Urdu mein short wazahat hai ke woh section kya karta hai.

Pehle ye samajh lo (Roman Urdu mein)

- **Prompt ko 6 phases mein todo.** Itna bada tool ek hi jawab mein poora nahi aa sakta; AI beech mein code kaat deta hai. Isliye is prompt mein *Build Phases* diye hain. Har phase ke baad AI se kaho: "Phase 2 shuru karo."
- **Dashboard ka look:** Dark neon "AI template" nahi. Microsoft Fluent 2 + Google Material 3 jaisa saaf, enterprise-style layout: app bar, navigation rail, cards, data grid, side panels, tooltips.
- **Har cheez explain:** Har number (EV%, Vig, Kelly, xG) ke saath ek (?) info icon hoga jo simple zaban mein batayega ke yeh kya hai aur kaise nikla.
- **Haqeeqat:** Koi bhi model profit guarantee nahi karta. Tool sirf analysis aur discipline ke liye hai. Is baat ka disclaimer UI mein hona chahiye.

MASTER PROMPT (copy from here)

1. Role & Objective

You are a Principal Quantitative Sports Engineer and Senior Full-Stack Developer (Python + modern frontend). Build a complete, production-quality, self-hosted web application named **ApexQuant v10.0 – Football & Tennis Quantitative Analysis Suite**, created by **Muhammad Atif**.

The tool compares sharp bookmaker prices (Pinnacle) against soft bookmaker prices (1xBet), removes the bookmaker margin, blends the result with an independent statistical model, audits risk, and presents everything in a professional, fully explained dashboard.

Coding rules (strict):

- Write 100% complete, runnable, modular code. No placeholders, no `# TODO`, no truncated functions, no "rest of code here".
- Every formula must be fully implemented and documented with a docstring.

- Every external call must have timeouts, retries, caching and graceful fallback. One failing API must never crash the app.
- Add unit tests for all math functions (`tests/test_quant_engine.py`).
- Work in the phases defined in Section 11. Finish one phase completely before starting the next.

---

2. Tech Stack

- **Backend:** Python 3.11+, FastAPI, Pydantic v2, httpx (async), scipy, numpy, pandas, python-dotenv, duckduckgo-search, feedparser, Pillow.
- **Frontend:** Plain HTML + CSS + vanilla JavaScript (ES modules). No heavy framework required. Charts via lightweight Chart.js or hand-written SVG.
- **Storage:** Local CSV (`audit_memory_vault.csv`) plus a small SQLite cache for API responses.
- **Config:** `.env` file for keys (`ODDS_API_KEY` , `GEMINI_API_KEY` , `GROQ_API_KEY` , `SHEETS_WEBHOOK_URL`).

---

3. Data Sources (Free-first)

3.1 Fixtures (no key): ESPN public scoreboard JSON.

- Football leagues: `eng.1` , `esp.1` , `ita.1` , `ger.1` , `fra.1` , `uefa.champions` via `https://site.api.espn.com/apis/site/v2/sports/soccer/{league}/scoreboard` .
- Tennis: `.../tennis/atp/scoreboard` and `.../tennis/wta/scoreboard` .
- Extract: teams/players, start time (convert to **PKT, UTC+5**), venue, live score, status, records.
- Note: these are unofficial endpoints. Code defensively and handle missing fields.

3.2 Odds – Mode A (Automated): The Odds API (free tier, key in `.env`).

- Markets: 1X2/Moneyline, Asian Handicap/Spreads, Totals.
- Bookmakers: Pinnacle (sharp) and 1xBet (soft), where available in the selected region.
- Track the free-tier request quota, cache aggressively, and show remaining quota in the UI.
- If a bookmaker or market is unavailable, show "Not available" and suggest Mode B instead of inventing numbers.

3.3 Odds – Mode B (Screenshot OCR):

- Two side-by-side drag-and-drop zones: **1xBet (Soft)** and **Pinnacle (Sharp)**, with click-to-zoom preview.
- Parse with Gemini Flash (free tier) or Groq Vision.
- **Strict OCR lock:** extract only text visibly present in the image. Never guess or fill a missing line.
- Compare both screenshots and flag `Line Mismatch` (for example AH -0.5 on one side vs AH -0.75 on the other).
- Show extracted values in an editable table so the user can correct OCR mistakes before analysis.

3.4 News & Injuries (no key): `duckduckgo-search` news plus Google News RSS. Fetch confirmed injuries, suspensions, lineup leaks, press-conference quotes and H2H context for the match date. Keep headline, source, timestamp and URL for every item.

3.5 Weather (no key): Open-Meteo forecast API using venue coordinates. Fetch for match hour: temperature, wind speed, humidity, precipitation probability. Explain the effect on scoring pace and ball physics. Handle indoor venues and roofed courts.

---

4. Quantitative Engine (`backend/quant_engine.py`)

4.1 Shared market math

1. **Margin (Vig):** `Vig = (Σ 1/O_i) - 1` for the sharp book.
2. **De-vigging:** Primary method is the **Power Method**: solve `Σ (1/O_i)^k = 1` with `scipy.optimize.brentq`. Fall back to the **Multiplicative** method if the solver fails. Show both results in the UI.
3. **Expected Value:** `EV% = (P_true × O_soft − 1) × 100`.
4. **Quarter-line Asian Handicap (±0.25, ±0.75):** split the stake 50/50 across the two adjacent lines (for example −0.25 = half on 0.0 / Draw No Bet and half on −0.5). Handle push probability, half-win and half-loss exactly. Include unit tests with hand-checked examples.
5. **Kelly sizing:** Quarter-Kelly `f* = max(0, 0.25 × ((P × O) − 1) / (O − 1))`, expressed as % of bankroll, with a hard cap (default 2%) configurable in settings.

4.2 Verdict tiers

- 🟢 **SNIPER BET:** EV ⩾ +1.20% and all veto rules pass.
- 🟡 **CAUTION (small stake):** EV from +0.50% to +1.19% with supportive model/news. Suggest a flat 0.50% stake.
- 🔴 **VETO (skip):** EV < +0.50% or any veto rule fires.

4.3 Football: Dixon-Coles + xG

- Compute home λ and away μ from attack strength, defence weakness, recent xG and home advantage.
- Apply the Dixon-Coles low-score correction τ with ρ ≈ −0.13:
  - τ(0,0) = 1 − λμρ
  - τ(1,0) = 1 + μρ
  - τ(0,1) = 1 + λρ
  - τ(1,1) = 1 − ρ
  - τ = 1 for all other scores
- Build a 10×10 scoreline probability matrix and renormalise. Derive 1X2, Asian Handicap (all lines), Over/Under (all lines) and BTTS from it.
- **Blend:** `P_master = 0.70 × P_pinnacle_devigged + 0.30 × P_model`. Make the weights configurable in settings.
- If xG data is unavailable from free sources, clearly mark the model as "low-confidence (league-average inputs)" instead of hiding it.

4.4 Tennis: Markov chain + surface Elo

- **Barnett-Clarke hierarchy:** point → game → tiebreak → set → match (Best-of-3 and Best-of-5). Inputs are each player's service-point win probability, adjusted for opponent return strength: `p_adj = p_A + (r_avg − r_B)`.
- Output exact hold probability, tiebreak probability, set-score distribution (6-0 … 7-6), match win probability, expected total games, and game-handicap and Over/Under distributions (for example O/U 21.5, 22.5 and ±2.5, ±3.5).
- **Elo:** 50% overall Elo + 50% surface Elo (Hard, Clay, Grass, Indoor Hard).
- **Extras:** Dominance Ratio (return points won % ÷ serve points lost %), break-point conversion/save delta, and a fatigue penalty from minutes and sets played in the last 72 hours.

---

5. Risk Governance & Audit (`backend/auditor.py`)

Every match goes through four veto rules:

Rule	Name	Trigger
VR-01	Minimum Edge Gate	Max EV < +0.50% = Veto. ≥ +1.20% = Pass. In between = Caution.

Rule	Name	Trigger
VR-02	Late Injury / Steam Trap	Soft odds look high only because live news shows a key absence, injury or medical timeout.
VR-03	Liquidity & Variance Cap	Pinnacle vig > 4.80% or soft odds > 4.50.
VR-04	Model vs Market Divergence	Model disagrees with the Pinnacle-implied pick by more than 8.0 percentage points.

Error classification tags: `EC-00 Clean Setup`, `EC-01 Line Mismatch`, `EC-02 Injury/Steam Trap`, `EC-03 High Vig/Low Liquidity`, `EC-04 Model/Tactical Conflict`.

Memory Vault: append every audited match to `audit_memory_vault.csv` (timestamp, match, market, odds, probabilities, EV, stake, verdict, EC tag, result column left empty for later settlement). Optional Google Sheets webhook. Also provide an **"Update result"** action so the user can record the outcome and the app can show a running ROI and closing-line-value (CLV) summary.

---

6. Raw Data Inspector & "Verify with Claude" Generator

At the bottom of every match analysis:

1. **Raw Data Inspector:** collapsible tabs (Odds, Model Parameters, Weather, News, Raw JSON) with exact timestamps and source URLs.
2. **Verification prompt box** with a **"Copy Prompt to Verify in Claude"** button. The generated prompt must be self-contained, include all fetched data for that match, and state:

> "Act as an independent senior quantitative sports syndicate auditor. Below is the complete real-time data fetched by my ApexQuant v10.0 tool for [Match Name] (Sharp Pinnacle odds, soft 1xBet odds, de-vigged probabilities, Dixon-Coles / Markov outputs, live weather, and latest injury/ground news). Independently verify every mathematical calculation, audit the tactical and injury impact, check whether any hidden trap exists, and confirm in Roman Urdu whether I should execute this bet, reduce stake, or veto it."

---

7. UI Design System – Microsoft Fluent 2 + Google Material 3 style

Important: The interface must look like a real enterprise product (Microsoft 365 admin center, Azure Portal, Google Analytics, Google Search Console), **not** a generic AI dashboard. No neon glows, no gradients on cards, no glassmorphism, no emoji-heavy headings, no purple-blue "AI" palette.

7.1 Visual language

- **Fonts:** `Segoe UI`, `Roboto`, `Inter`, system-ui (fallback stack). Numbers in `Roboto Mono` or `Consolas` with `font-variant-numeric: tabular-nums`.
- **Colours (light theme):**
  - App background `#F3F2F1` (Fluent neutral), surfaces `#FFFFFF`, subtle wells `#FAF9F8`
  - Borders `#E1DFDD`, text primary `#201F1E`, secondary `#605E5C`
  - Primary action `#0F6CBD` (Microsoft blue), secondary accent `#1A73E8` (Google blue)
  - Success `#107C10`, Warning `#BC4B09`, Error `#C50F1F` with soft tinted backgrounds for status chips
- **Shape & depth:** 4px corner radius (Fluent), 1px borders, elevation only on dialogs and menus (`0 8px 16px rgba(0,0,0,.14)`), cards stay flat.
- **Spacing:** 4px base grid (4/8/12/16/24).
- **Icons:** one consistent line icon set (Fluent UI icons or Material Symbols, inline SVG). No random emojis in navigation or tables.
- **Dark theme:** provide a proper dark theme toggle (`#1B1A19` base) following Fluent dark guidelines, not a neon one.
- **Accessibility:** WCAG AA contrast, visible focus rings, full keyboard navigation, `aria` labels.

7.2 App shell

1. **Top app bar (48px, Microsoft 365 style):** product name "ApexQuant", global search, live PKT clock, data-source status chips (ESPN, Odds API, Weather, News, OCR) with green/amber/red dots, theme toggle, settings and help buttons.
2. **Left navigation rail (collapsible, 220px):** Overview, Football, Tennis, Screenshot Analyzer, Audit Vault, Settings, Glossary & Help.
3. **Command bar under the app bar** (Fluent CommandBar): Refresh, Filters, Export CSV, Copy Verification Prompt.
4. **Content area:** page title with breadcrumb, then cards and data grids.

7.3 Pages

Overview: KPI tiles (Matches today, Sniper bets, Caution, Veto, Vault ROI), then a sortable **Match Radar data grid** (columns: Time PKT, Competition, Match, Status/Score, Best market, EV%, Verdict chip). Row click opens the match detail. Filters: sport, league, verdict, EV ⩾ 1.2%.

Match Detail (tabbed, like Azure resource pages):

- **Summary tab:** verdict banner (chip + pick + EV% + quarter-Kelly stake), fixture info, weather chips.
- **Odds & Value tab:** clean comparison grid for 1X2/Moneyline, Asian Handicap/Spread, Totals with columns: 1xBet odds, Pinnacle odds, movement arrow, de-vigged probability, model probability, EV%. Toggle between Auto-fetched and Screenshot mode.
- **Model tab:** probability comparison bars (Pinnacle vs Model vs 1xBet implied), λ/μ or hold/break breakdown, scoreline heatmap or set-score table.
- **Ground Reality tab:** weather impact, surface/pitch notes, verified injuries and news with source links, plus the **Roman Urdu analysis** in four labelled parts: (1) Price value and math logic, (2) Ground reality, pitch/surface and weather, (3) Verified injuries and news, (4) Final one-line action.
- **Risk Audit tab:** VR-01 to VR-04 checklist with pass/fail and the reason, EC tag, "Save to Vault" button.
- **Raw Data tab:** inspector and verification prompt (Section 6).

Screenshot Analyzer: two upload cards, editable extracted-lines table, mismatch warnings, then "Run analysis".

Audit Vault: data grid of saved audits with search, filters, result entry, CSV export, ROI/CLV summary chart.

Settings: API keys status, region, bankroll, Kelly fraction and cap, blend weights, thresholds, theme.

7.4 "Explain everything" requirement

- Every metric label has a **(?) info icon** opening a Fluent-style teaching callout with: plain-language meaning, the formula, and a worked example using the current match's numbers.
- Every verdict has a **"Why this verdict?"** side panel listing each rule and number that led to it.
- A **Glossary & Help** page explains: vig, de-vigging, EV, Kelly, Asian Handicap quarter lines, xG, Dixon-Coles, Markov chain, Elo, CLV, line movement.

- Empty, loading (skeleton shimmer, no spinners-only), and error states must be designed with a helpful message and a retry button.
- Footer disclaimer: "Analytical tool only. No outcome is guaranteed. Bet responsibly and only where legal."

---

8. Backend API Endpoints (`backend/main.py`)

- `GET /api/fixtures?sport=football|tennis&date=YYYY-MM-DD`
- `GET /api/odds/{match_id}` (Mode A)
- `POST /api/ocr` (two images, Mode B)
- `POST /api/analyze` (runs math, news, weather, audit; returns full structured result)
- `GET /api/health` (status of each data source and API quota)
- `GET/POST /api/vault` and `PATCH /api/vault/{id}` (list, save, settle)
- `GET /api/vault/export` (CSV download)
- Serve `frontend/` as static files; enable CORS for local use; structured error responses.

---

9. Project Structure

```
apexquant/
├─ requirements.txt
├─ .env.example
├─ README.md            (setup, run, troubleshooting, screenshots guide)
├─ backend/
|  ├─ config.py
|  ├─ schemas.py        (Pydantic v2)
|  ├─ data_fetcher.py
|  ├─ quant_engine.py
|  ├─ auditor.py
|  └─ main.py
├─ frontend/
|  ├─ index.html
|  ├─ styles.css        (design tokens as CSS variables)
|  └─ app.js            (split into ES modules if large)
└─ tests/
   └─ test_quant_engine.py
```

---

10. Quality & Honesty Rules

- Never fabricate odds, injuries or statistics. If data is missing, show "Data unavailable" and lower the confidence badge.
- Show data freshness (timestamp) next to every number that comes from an API.
- Roman Urdu report must only use facts present in the fetched data and cite which section each claim comes from.
- Add a confidence indicator (High / Medium / Low) based on data completeness.
- Never present a bet as guaranteed; wording like "model-estimated edge" is required.

---

11. Build Phases (reply with one phase at a time)

1. **Phase 1:** `requirements.txt`, `.env.example`, `config.py`, `schemas.py`, project README.
2. **Phase 2:** `quant_engine.py` complete with tests.
3. **Phase 3:** `data_fetcher.py` complete (ESPN, Odds API, Open-Meteo, news, OCR).
4. **Phase 4:** `auditor.py` and `main.py`.
5. **Phase 5:** `frontend/styles.css` (design tokens, components) and `index.html` (shell and pages).
6. **Phase 6:** `frontend/app.js` (all logic, charts, copy-prompt button, info callouts), then a final integration checklist.

At the end of each phase, list what was completed and what the next phase needs.

---

12. Start

Begin with **Phase 1** now. Write full code only; keep explanations short.

END OF MASTER PROMPT

Mere suggestions (Roman Urdu mein)

1. **Phase-by-phase chalao.** Ek hi message mein sab kuch maangoge to code adhoora aayega.
2. **Free odds ka masla:** The Odds API ke free plan mein 1xBet hamesha available nahi hota aur monthly quota kam hota hai. Isliye Mode B (screenshot) rakhna sahi faisla hai.
3. **Model ki limit:** Free data se xG aur player stats kam milte hain, isliye 30% model weight ko settings mein adjustable rakha hai.

4. **Test zaroor karo:** Quarter-line Asian Handicap ka math sabse ghalti-wala hai; isliye unit tests prompt mein shamil kiye hain.
5. **Responsible rahein:** Quarter-Kelly aur 2% cap ka matlab hai bankroll ko bachana. Tool profit ki guarantee nahi deta.